SW Engineering CSC648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 2 Part I

History Table	
Date Submitted	April 03, 2026
Date Revised	

Table of Contents

1. **Executive Summary . . . . . pg#**
2. **Main Data Items and Entities . . . . . pg#**
3. **Functional Requirements . . . . . pg#**
4. **UI Storyboards . . . . . pg#**
5. **High Level Architecture/Database Organization . . . . . pg#**
6. **Key Risks . . . . . pg#**
7. **Project Management . . . . . pg#**
8. **Use of GenAI Tools . . . . . pg#**
9. **Team Lead Checklist . . . . . pg#**

1. Executive Summary:

College students often have so much stuff towards the end of their four years and even semesters. These items include textbooks from past semesters, small furniture, electronics, or school supplies. At the same time, many other students are looking for affordable alternatives instead of buying new items. However, existing marketplaces such as general online classifieds are not designed specifically for university communities and can feel inconvenient or unsafe for student use. Having to meet up in locations that aren't always safe, for example Facebook marketplace, etc. GatorBazaar is a marketplace platform designed specifically for the San Francisco State University (SFSU) community where students and professors can easily buy, sell, and exchange items within a trusted campus environment.

GatorBazaar provides a simple web-based platform where users can list items they no longer need and browse items posted by others. Listings are organized into categories such as textbooks, electronics, clothing, and dorm essentials, and many other features we have coming down the pipeline, making it easy for users to quickly find what they are looking for. Students can post items for sale, upload descriptions and prices, and connect with buyers within the SFSU community. Professors can also use the platform to share or sell academic materials or recommend resources to students. By centralizing these exchanges in one place, GatorBazaar helps reduce waste, saves students money, and creates a convenient and safe way to reuse items within the campus community.

As San Francisco State University students ourselves, our team is dedicated to making a marketplace that not only our fellow students feel confident and safe using, but one that we

would use as well. With student safety and convenience at our highest priority, we believe we can make GatorBazaar the ideal cornerstone for second-hand student shopping.

2. Main data items and entities:

- **Admin Users** - Users that manage reports received by either listings or users. Able to issue warnings to users about violations or take action in the form of removing a listing or banning a registered user. Reports will be displayed on the Registered Users profile or Listing Page.
- **Unregistered Users** - Users who can only access the homepage, browse, search, and filter listings, and view listing details. Do not need to log in or register.
- **Registered Users** - Users that have signed up on the site using a SFSU verified email.
    - **Buyers** - Registered users looking to purchase listings on the site. All registered users are treated as buyers innately.
        - **Sent Request** - Used to show a buyer's interest in a listing that is sent to the seller.
        - **Sent Request List** - A list showing all items a buyer has sent a request to buy (seen by buyer only).
        - **Bought Listings List** - A list of listings previously purchased by a buyer.
    - **Sellers** - Users that have at least one listing on the site.
        - **Listing -** An item for sale posted by and belonging to a registered user.
            - **Title -** The name of the item for sale.
            - **Image** - An image of the item for sale set by the seller.
            - **Description -** A description of the item for sale set by the seller.
            - **Price** - Price of the item set by the user.
            - **Category** - Grouping of a listing provided by the site.
            - **Tags** - Additional searchable terms set by the seller.
                - **Course** - Sellers can also set an SFSU course as a tag for listings related to a course.
            - **Pending Request** - A request received by the seller used to show a buyer's interest in the listing.
            - **Listing Report** - A report specific to a listing sent by other users if the listing breaks the guidelines of the site (seen by admin)

            - **Active -** Shows whether listing is currently being sold or not (system purposes)
        - **Active Listing List** - A list of all listings a seller currently has on the user's page (seen by all users)
        - **Sold Listing List** - A list of all past listings sold by a seller on the user's page (seen by user owning the list and admins)
    - **Messaging** - Allows communication between the seller and the buyer.
        - **Message** - A communication instance between registered users.
        - **Inbox** - A list of current and past messages between registered users. Allows registered users to send messages to other registered users.
        - **Notification** - When a registered user receives a message from another registered user, their inbox will indicate that they have a new message.
    - **Meet up** - A tracked location on campus where the buyer and seller agree to enact the exchange.
    - **User Rating** - Aggregate rating given to a seller/buyer by other users (seen by all users)
    - **User Report** - Report specific to a registered user sent by other registered users if guidelines of the site are broken by the reported user. (seen by admins)
    - **Email Verification** - Checks whether a user attempting to register has a valid SFSU email.
- **Search** - A method of filtering listings by their attributes(name, tags, price, etc.)
    - **Roommate Search** - Searching listings within the apartment category to find roommates.

3. Functional Requirements

1 - must have | 2 - desired | 3 - opportunistic (temp rating below)

Priority 1

Unregistered User

The system shall allow visitors to access the marketplace homepage.

The system shall allow visitors to browse publicly available item listings.

The system shall allow visitors to search listings using keywords.

The system shall allow visitors to view item details including title, description, price, and images.

The system shall allow visitors to filter listings by category.

Registered User

The system shall allow users to register using an SFSU email address.

The system shall allow users to receive notifications when they receive messages.

The system shall verify that registered users are associated with an SFSU email domain.

Seller

The system shall allow registered users to create listings for items they wish to sell.

The system shall allow users to upload images when creating listings.

The system shall allow users to delete their listings.

Buyer

The system shall allow users to initiate the sending of messages to other users regarding listings.

Admin

The system shall allow administrators to remove listings that violate policies.

The system shall allow administrators to monitor marketplace activity.

Priority 2

Unregistered User

Registered User

The system shall allow users to report listings that violate platform rules.

The system shall allow users to give a rating to either seller or buyer.

Seller

The system shall allow users to edit their listings.

The system shall allow users to mark listings as sold.

Buyer

Admin

The system shall allow administrators to review reported listings.

The system shall allow administrators to manage listing categories.

Priority 3

Unregistered User

Registered User

Seller

Buyer

Admin

The system shall allow administrators to suspend users who repeatedly violate rules.

To Decide:

The system shall allow users to report listings that violate platform rules. [2]

The system shall allow administrators to review reported listings. [2]

↑↑↑ What Priority should we set these as, 1 or 2? We mention them in a Use Case, so should it be P1?

The system shall allow users to edit their listings. [2]

↑↑↑ What Priority for these too, editing after posting is a nice feature, but worst comes to worst they can simply make another listing, and its not mentioned in Use Cases

The system shall allow users to delete their listings. [1]

The system shall allow users to mark listings as sold. [2]

↑↑↑ For these features, which should we prioritize, it we keep delete as P1 then its fine to leave editing as P2, but if not then editing probably also must be put as a P1

Non-Functional Requirements

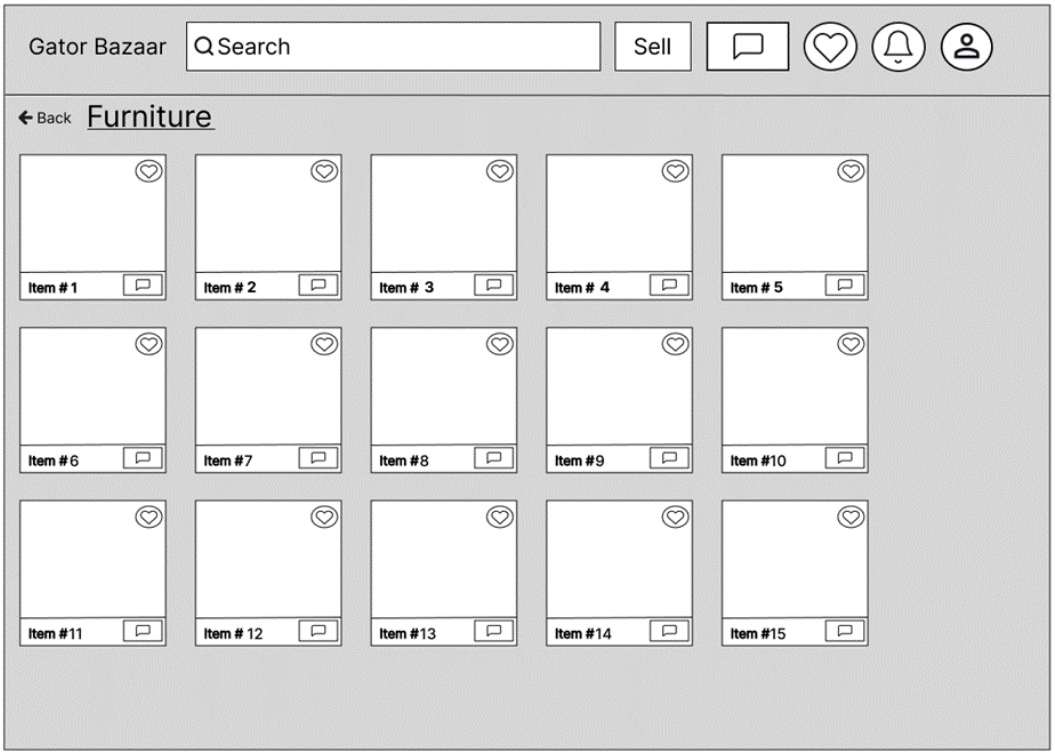
1. Application shall be developed, tested and deployed using tools and cloud servers approved by Class CTO and as agreed in M0
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. All our selected application functions shall be rendered well on mobile devices (no native app to be developed)
4. Posting sales information and messaging to sellers shall be limited only to SFSU students
5. Critical data shall be stored in the database on the team's deployment server.
6. No more than 50 concurrent users shall be accessing the application at any time
7. Privacy of users shall be protected
8. The language used shall be English (no localization needed)
9. Application shall be very easy to use and intuitive
10. Application shall follow established architecture patterns
11. Application code and its repository shall be easy to inspect and maintain
12. Google analytics shall be used
13. No e-mail clients or chat services shall be allowed. Interested users can only message to sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application
14. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items
16. Media formats shall be standard as used in the market today
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Spring 2026. For Demonstration Only" at the top of the WWW page Nav bar. (Important to not confuse this with a real application). You must use this exact text without any editing.

4. UI Storyboards

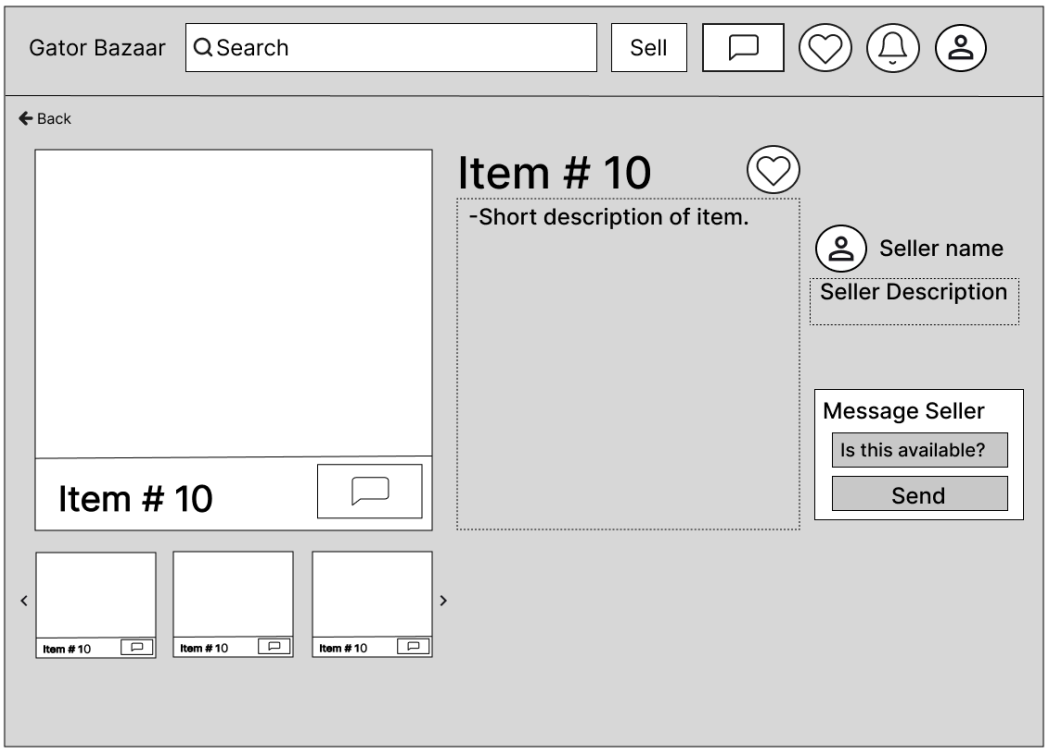
Use Case 1: Buying Furniture in Gator Bazaar



Landing on the home page of our application, John navigates to the furniture tab located in Popular Categories at the top of the page.

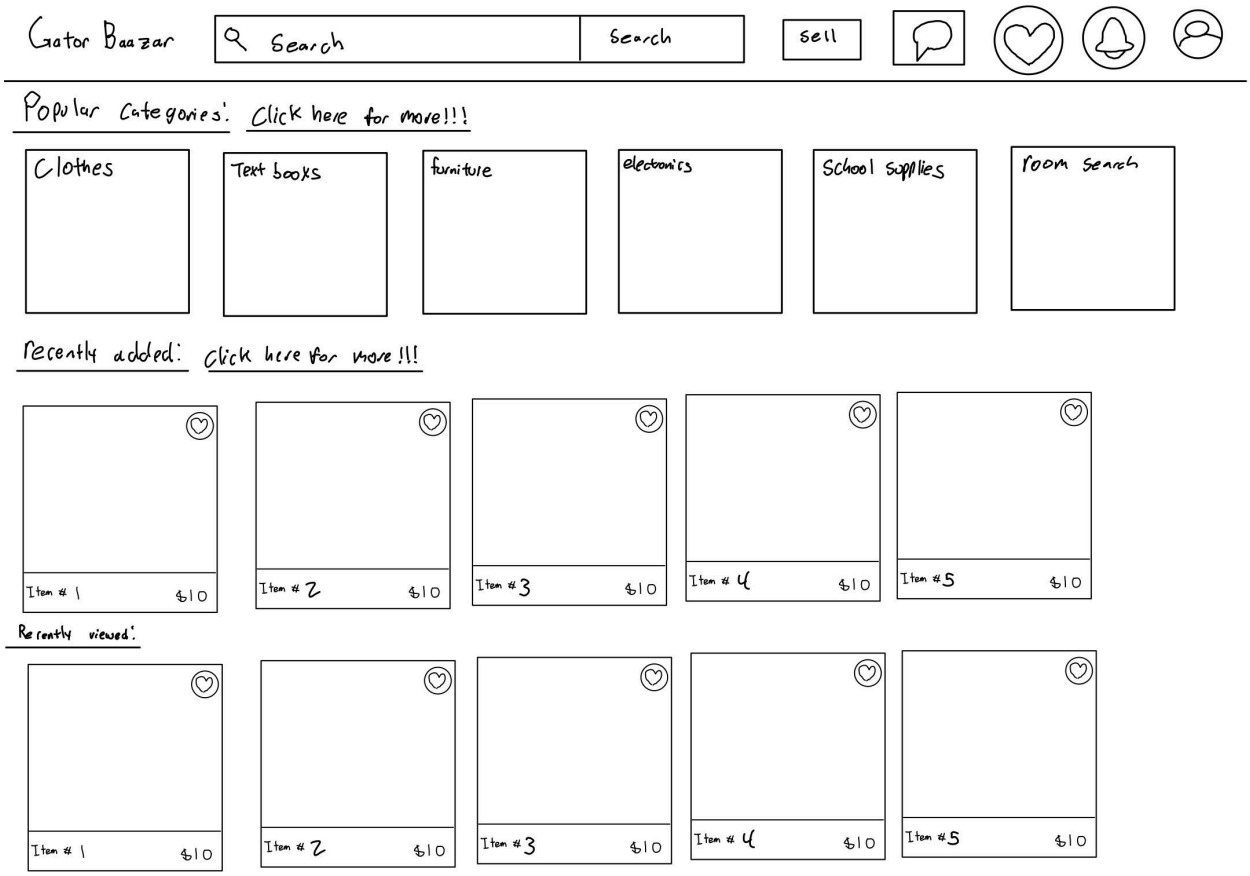


Then he can start scrolling through the page to find lists of furniture he needs.

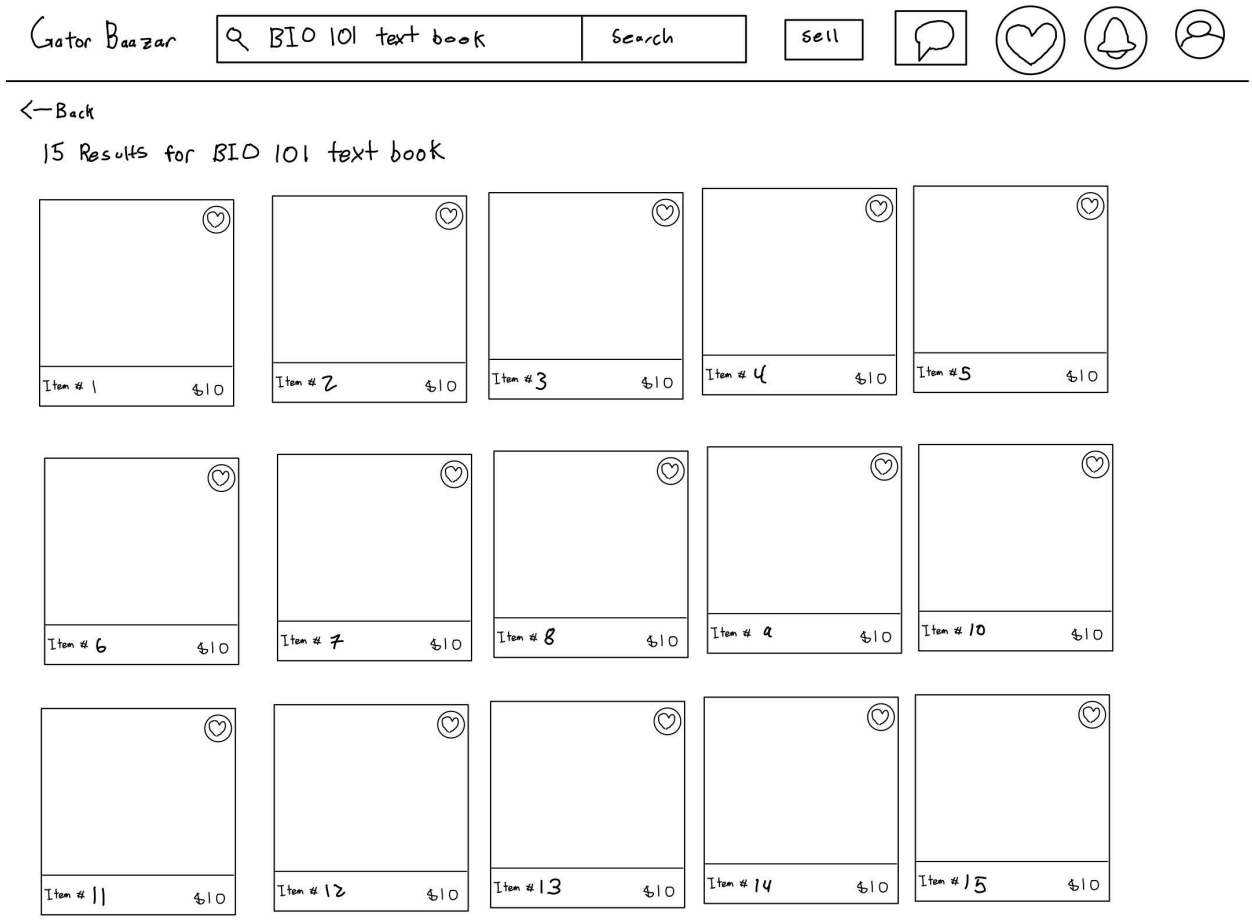


Once he clicks on a furniture listing he likes, he will be brought to the 'buying' tab where he can find the description, and other useful images of the furniture from the seller. He can either like to save the furniture or message the seller to buy the furniture from the seller at the top right of the listing.

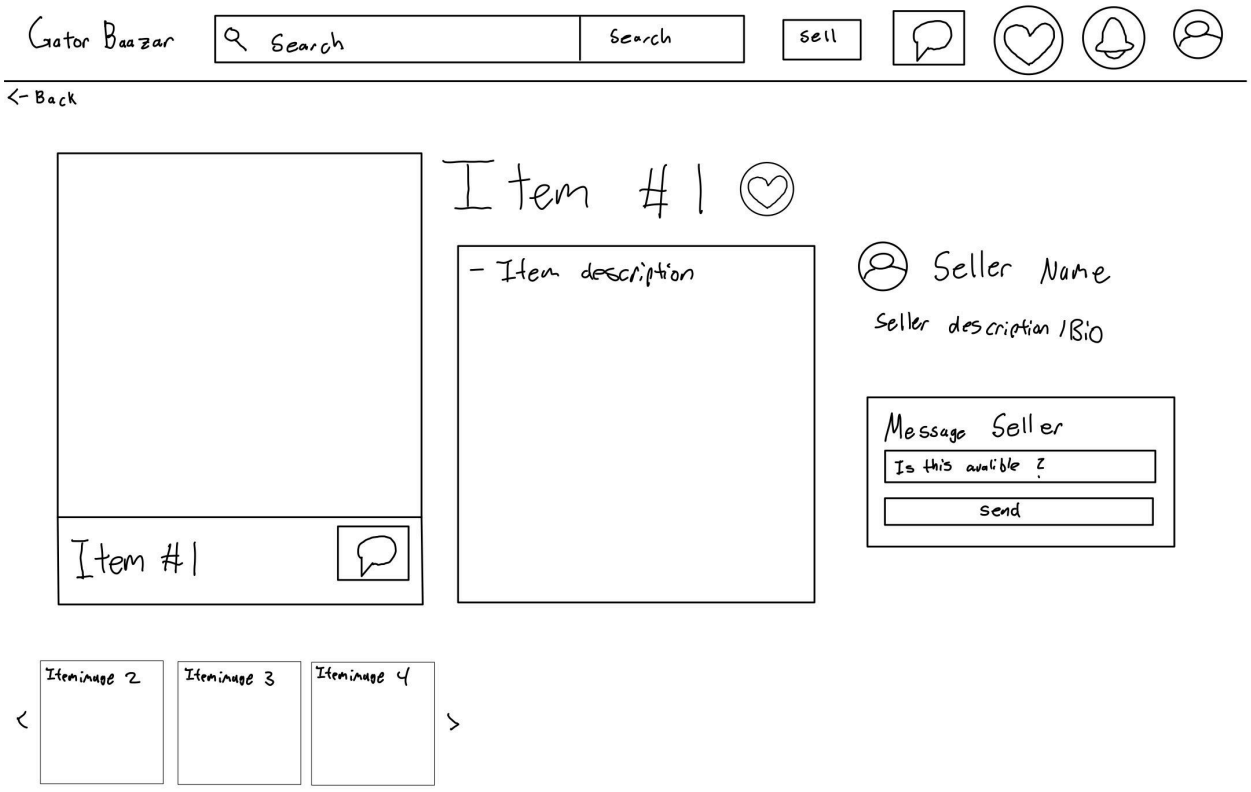
Use Case 2: Buying text books



Jose Is looking to buy some text books for his classes so he logs into gator bazaar and searches for his Bio 101 textbook

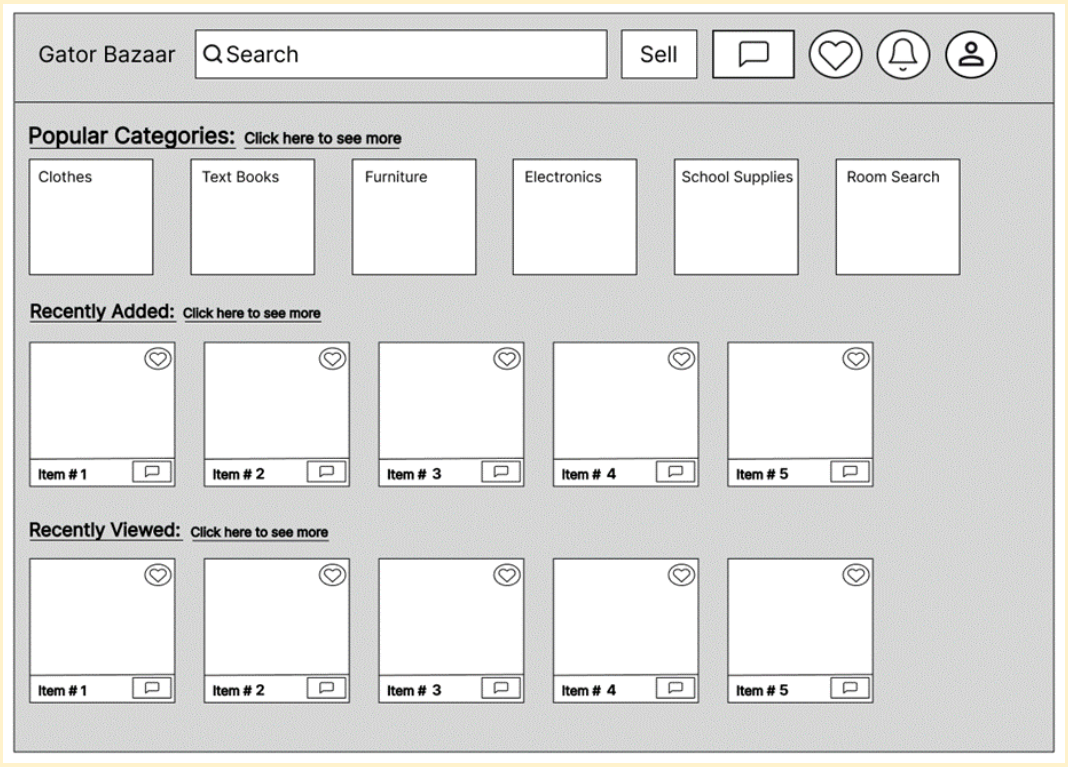


Once Jose searches for his textbook he is prompted with the closest results to his given search

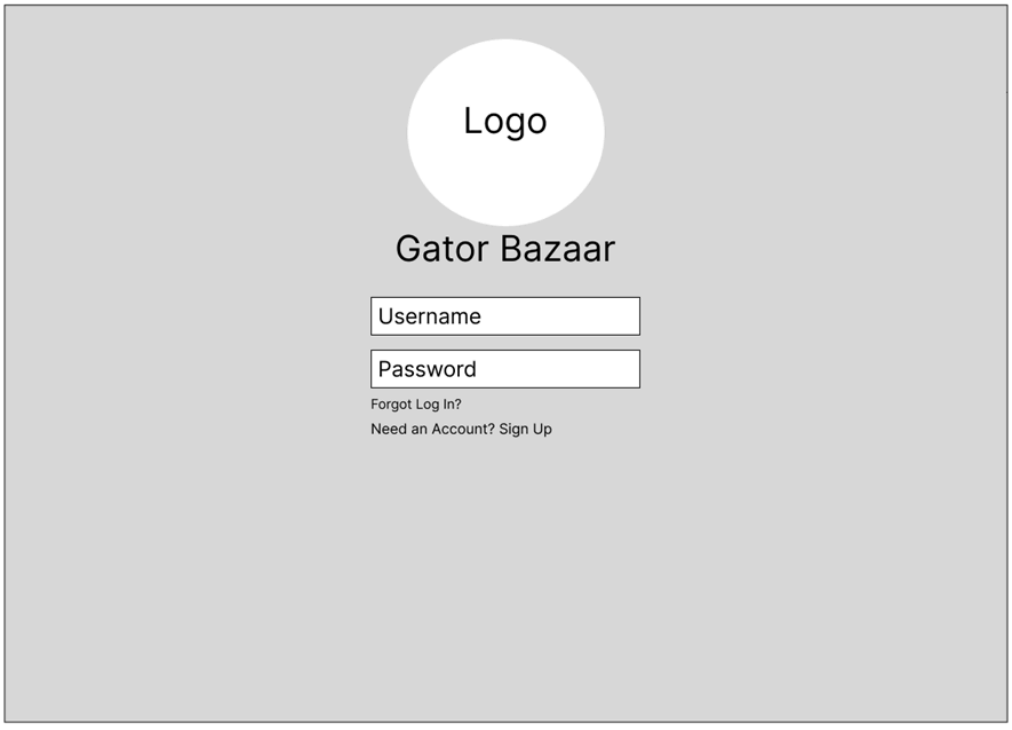


Once he selects a listing, jose can see the sellers item description, multiple images and the sellers bio. In order to be able to buy an item, Jose has to get in touch with the seller by sending them a message that he can send through the send message button.

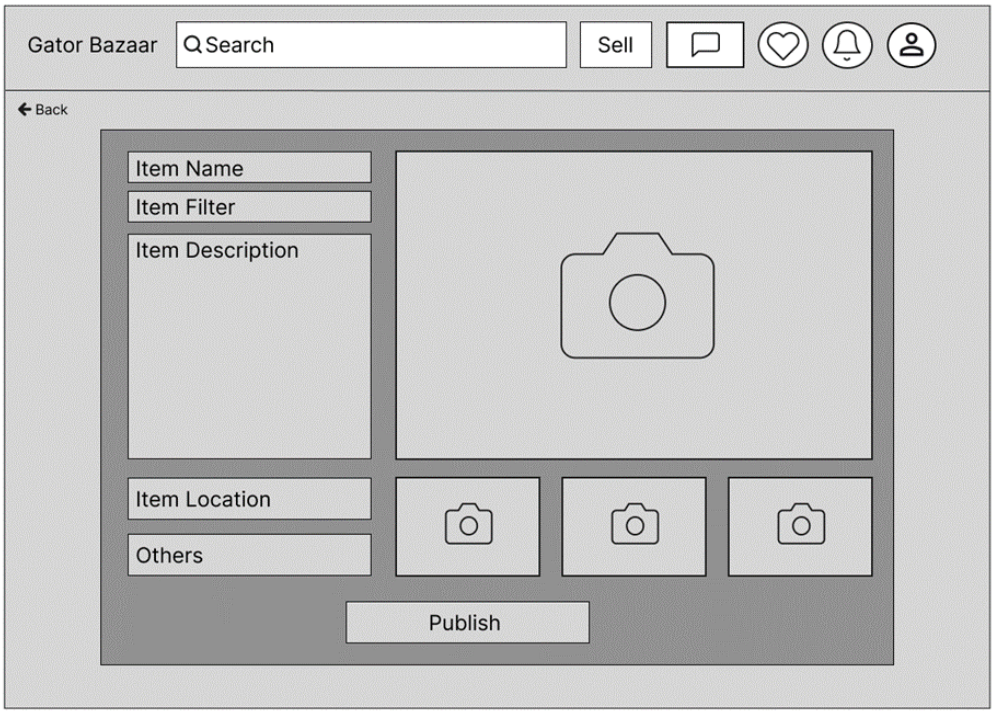
Use Case 3: Selling Items in Gator Bazaar



Javier wants to sell some of the things he does not need before moving out of his apartment. However, before he can sell items In Gator Bazaar he needs a SFSU-verified account using his SFSU email, which is the user log-in located at the top right of the page.



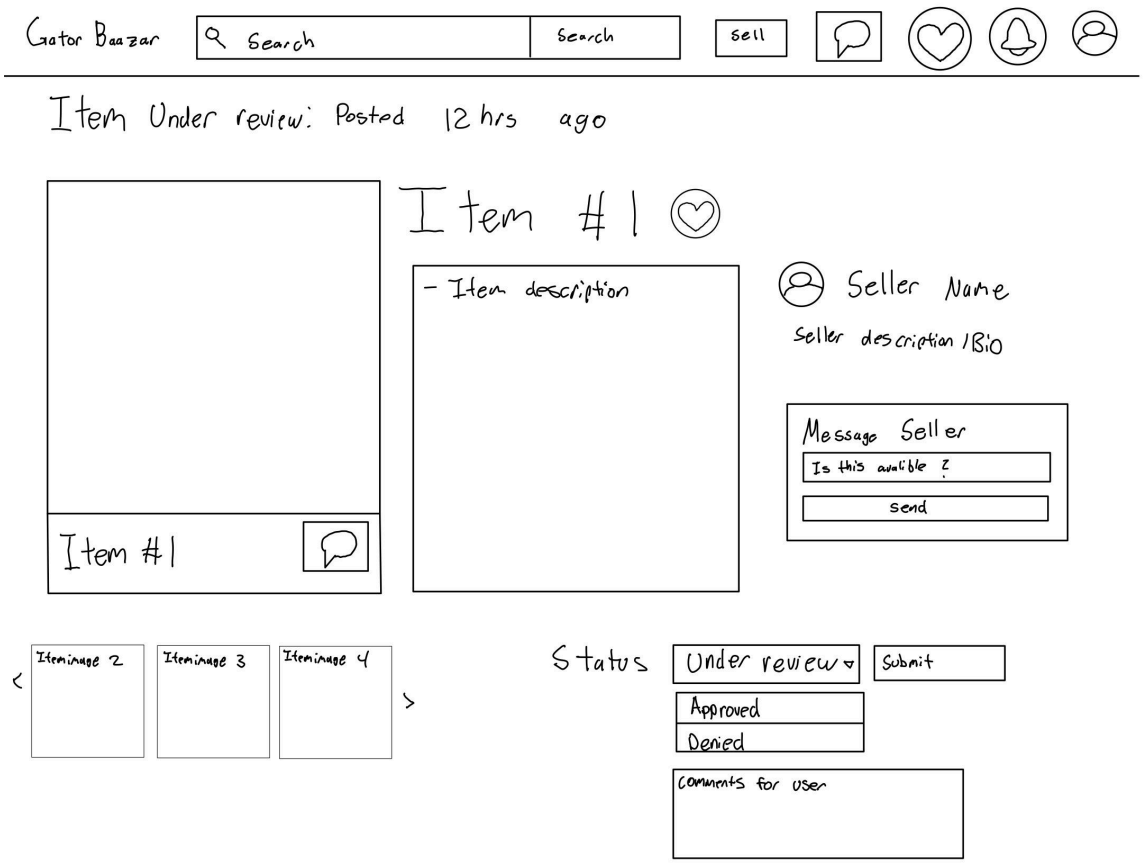
At the Log-in page he can sign in if he has an account set up, but he doesn't have one, so he needs to sign up with his SFSU verified account. Then he can log in with that account to start selling.



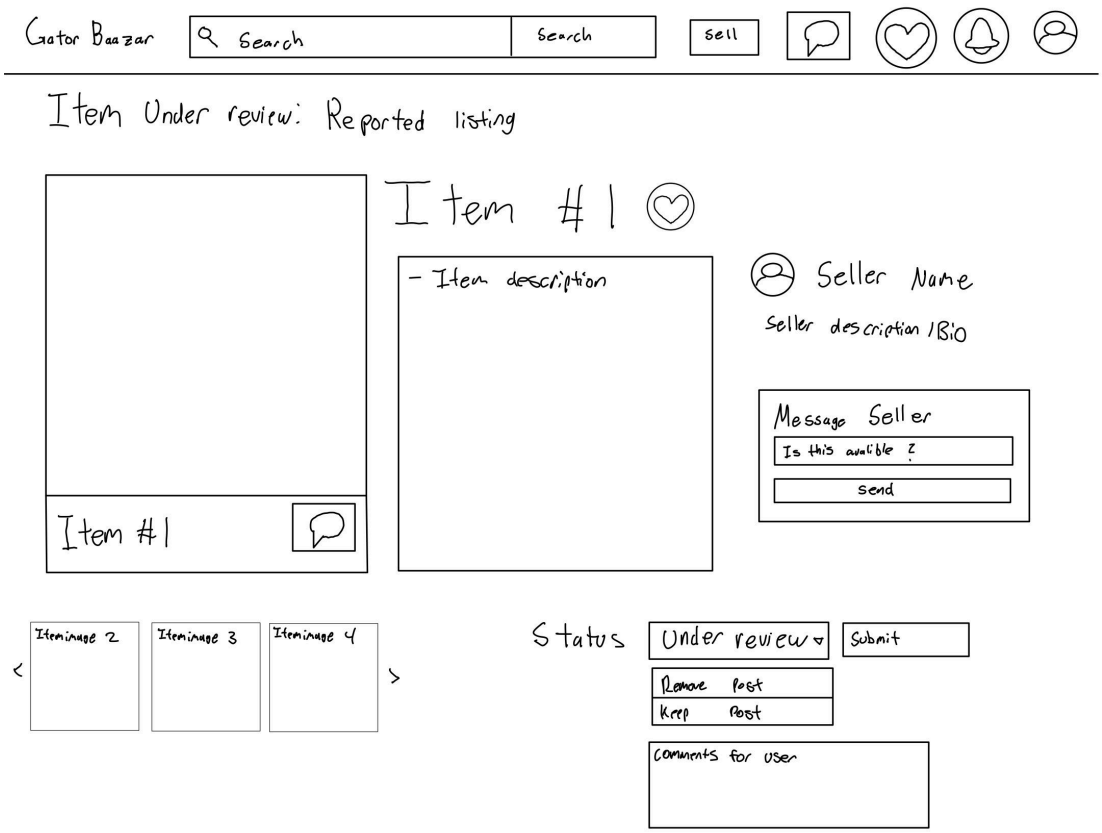
Once he signed up, he can click at the sell button at the top of the page to start selling. In the selling page, he can put the name, what type of item he's selling, the description, the location of the item, and other filters or necessary information he wants, and also upload necessary pictures of the item. When he is finished he can press the publish button to publish the listing to Gator Bazaar.

Case 5: User Admin

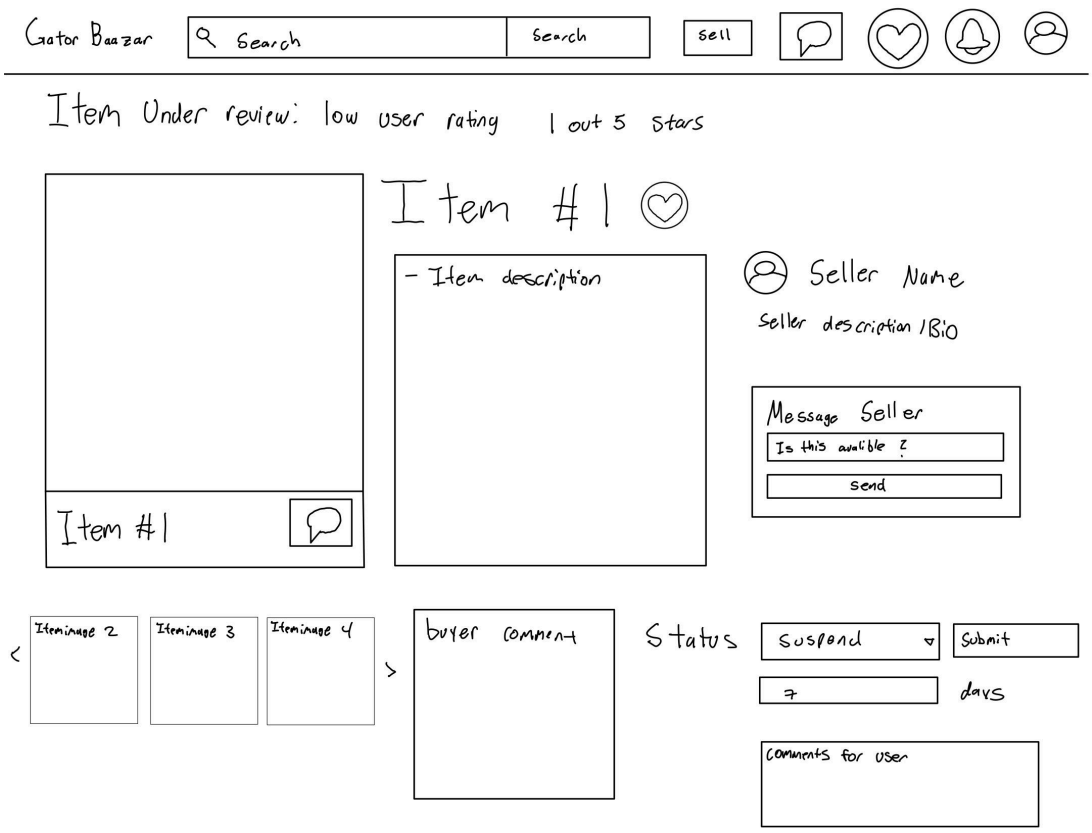
As a user Admin, Khalid has the ability to review different listings for different reasons.



In this case Khalid is reviewing a listing that was just posted. Here Khalid can approve or deny the post and give the user a reason why he didn't approve the post if the post needs further review.



Here Khalid is reviewing a post that was flagged/ reported by other users so here he can either remove the post or keep the post based on his review.



Here Khalid is reviewing a post with a low user rating. Here Khalid can read why a user gave this post a low review. Based on the post and review, Khalid can suspend the account for breaking guidelines deciding to ban the user for a week.

5. High Level Architecture/Database Organization:

Software stack:

- Server Host: AWS EC2 t3.small
- Operating System: Ubuntu 24.04.1
- Database: MySQL v 8.0.45
- Web Server: Apache 2.4.58
- Server-Side Language: JavaScript

Additional Technologies:

- Web Framework:
  - Express.js v 5.2.1
  - Node.js v 22.12.0
- Web Analytics: Google Analytics
- SSL Cert: Lets Encrypt (Cert Bot)
- React: 19.2.0

Browsers Supported:

- Google Chrome version 142 and later
- FireFox version 146 and later

Database Organization

Architecture Pattern (MVC)

The application follows the Model-View-Controller (MVC) pattern:

- Model:
  MySQL database schema and backend data logic
- View:
  React.js frontend interface rendered in the browser
- Controller:
  Node.js / Express.js backend handling API routes, business logic, and database interaction

Search/filter architecture and implementation

Our search Algo will be SQL-based keyword search %Like. Queries will be done through node.js/expressbackend. An example will consist of **WHERE title LIKE '%keyword%'** LIKE operator. Searchable fields consist of LISTINGS: title, description, category, course - TAGS tag_name. No current plan on switching from the recommended LIKE operator.

Registered Users Tables

- user_id(Primary Key)
- Email
- Password_hash
- role(buyer-seller/admin)
- Created_at
- User_rating
- is_suspended(boolean)

Listings Table

- listing_id(Primary Key)
- User_id (Foreign Key -> registered users)
- Title
- Description
- Price
- Category
- active(boolean)
- created_at

Listing Images Table

- Image_id (primary Key)
- Listing_id (foreign Key -> Listings)
- Image_url

Tags Table

- Tag_id(Primary Key)
- Tag_name

Listing Tags Table

- Listing_id Foreign Key-> Listing
- tag_id (Foreign Key → Tags)

Sent Requests Table

- request_id (Primary Key)
- listing_id (Foreign Key → Listings)
- buyer_id (Foreign Key → Registered Users)
- seller_id (Foreign Key → Registered Users)
- status (pending / accepted / rejected)
- created_at

Messages Table

- message_id (Primary Key)
- request_id (Foreign Key → Sent Requests)
- sender_id (Foreign Key → Registered Users)
- receiver_id (Foreign Key → Registered Users)
- message_content
- sent_at

Notifications Table

- notification_id (Primary Key)
- user_id (Foreign Key → Registered Users)
- message_id (Foreign Key → Messages)
- is_read (boolean)
- created_at

Bought Listings Table

- purchase_id (Primary Key)
- listing_id (Foreign Key → Listings)
- buyer_id (Foreign Key → Registered Users)
- purchase_date

Listing Reports Table

- report_id (Primary Key)
- listing_id (Foreign Key → Listings)
- reported_by_user_id (Foreign Key → Registered Users)
- reason
- created_at

User Reports Table

- report_id (Primary Key)
- reported_user_id (Foreign Key → Registered Users)
- reported_by_user_id (Foreign Key → Registered Users)
- reason
- created_at

Categories Table

- category_id (Primary Key)
- category_name

Non-Trival Algorithms / Processes

PENDING

6. Key Risks

1. Skills Risk

Some team members may have a limited amount of experience with full-stack development in a group project. Particularly with integrating the React front-end with the Node.js/Express backend and MySQL database. This could slow development and lead to implementation issues and troubles.

Mitigation:

Team members will divide work based on strengths (front-end & back-end) and actively learn missing skills through documentation, tutorials, and help from other team members. Regular check-ins and reviews will also help to ensure that knowledge is properly shared and understood across the whole team.

2. Schedule Risk

The project includes multiple features such as listings, messaging, user ratings, and admin controls, which may be difficult to complete within the given timeline if not managed properly.

Mitigation:

Our team will prioritize core functionality first and treat additional features as secondary. Tasks will be tracked using ClickUp with clear deadlines and responsibilities to mitigate potential issues whenever possible.

3. Technical Risk

Handling image uploads for listings may introduce some challenges not originally prepared for such as storage limitations or potentially slow loading times with multiple images.

Mitigation:

Our team will use good practice techniques in order to handle image handling issues, and possibly use other storage solutions if needed. Along with image size limits that will be enforced in order to keep storage issues to a minimum.

4. Teamwork Risk

Lack of clear communication regarding feature requirements may result in inconsistent implementation across components.

Mitigation:

The team will document requirements clearly and discuss them during meetings. Shared documentation will be maintained to ensure all team members understand what is required of a feature.

5. Legal / Content Risk

Users may post inappropriate or prohibited items (copyrighted material, or otherwise prohibited content), which could create compliance or safety concerns, and open the platform up to legal action if not dealt with.

Mitigation:

The system will include reporting functionality and admin moderation tools to remove inappropriate listings. Clear platform guidelines will be established, and admins will actively monitor reported content.

7. Project Management

Our process for task management in Milestone 2 was to separate Part I and II, and then divide the deliverables among the team. When creating our tasks, we initially separated the M2 Part I deliverable by section to get a good idea of what could be worked on independently. Then, we looked through each section to see what could be left as a whole task and what needed to be divided among multiple team members or divided into multiple tasks. While team members have input into what tasks they're assigned, we noted that certain tasks would be better off assigned to the front-end or back-end team. An example of this is the UI Storyboards, which were split among the front-end, or the Database Summary, which was assigned to the back-end lead. For Part II, we plan on taking the suggestion of sacrificing a front-end and back-end member to build the prototype and web pages, but we also emphasized that while only two members might be working on the Vertical Prototype, because we're defining the architecture and how the front-end and back-end will integrate, all team members must keep an eye on the process. We landed on a task due date of two days before the deadline for Part I and a week later for Part II, giving us sufficient time for revisions and peer reviews. Three days before the task deadline, we intend to have a checkup meeting where we can get a feel for where we are in terms of progress, with members already having posted occasional updates on their tasks.

8. Use of GenAI Tools:

9. Team Lead Check List:

Check List	
	So far all team members are fully engaged and attending team sessions when required
	Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing
	Team reviewed suggested resources before drafting Milestone 2
	Team lead checked Milestone 2 document for quality, completeness, formatting and compliance with instructions before the submission
	Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission
	Team shared and discussed experience with GenAI tools among themselves